
RACKLY

Auditoria Profunda del Codigo

Informe de Seguridad, Calidad e Integridad de Datos

Este informe presenta los resultados de una auditoria exhaustiva del aplicativo Rackly, abarcando ambas secciones (Pisos y Racks) y toda la infraestructura compartida. Se identificaron 11 vulnerabilidades criticas, 13 de alta severidad, 18 de severidad media y 15 de baja severidad en un total de 57 hallazgos. El analisis cubre logica de negocios, seguridad, manejo de errores, condiciones de carrera, atomicidad de transacciones, integridad de datos y calidad de codigo.

Fecha:	20 de junio de 2026
Branch:	main
Repository:	fileshidalgo-spec/rackly
Deploy:	rackly.pages.dev

Archivo: PisoSectoresTab, OcupacionTab, TrasladoTab, api.ts, sync-engine.ts, auth.ts, kardex.ts

1. Resumen Ejecutivo

Se realizo una auditoria profunda y exhaustiva del sistema de inventario Rackly, analizando la totalidad del flujo de operaciones en ambas secciones del aplicativo: la seccion de Pisos y la seccion de Racks. El alcance de la auditoria incluyo todas las funciones de movimiento (ingreso, salida, devolucion y traslado), la infraestructura compartida de autenticacion, sincronizacion offline, conexion a base de datos, y las definiciones de tipos de TypeScript. Se revisaron mas de 8,000 lineas de codigo en 12 archivos criticos del proyecto.

La auditoria revelo un total de 57 hallazgos distribuidos en cuatro niveles de severidad. El hallazgo mas preocupante es la exposicion de la clave de rol de servicio (service role key) de Supabase al navegador del cliente a traves de una variable de entorno publica (NEXT_PUBLIC_), lo cual permite a cualquier usuario eludir todas las politicas de seguridad a nivel de fila (RLS) y obtener acceso total de administrador a la base de datos. Este problema es critico y requiere atencion inmediata. Adicionalmente, se encontraron problemas graves en la logica de traslados que pueden generar stock fantasma y balances negativos, operaciones no atomicas que pueden dejar la base de datos en estado inconsistente, y multiples condiciones de carrera por doble clic que permiten operaciones duplicadas.

El siguiente cuadro resume la distribucion de hallazgos por seccion del aplicativo y nivel de severidad. Cada hallazgo fue clasificado considerando su impacto potencial en la integridad de los datos, la seguridad del sistema y la experiencia del usuario final. Los hallazgos criticos y de alta severidad requieren intervencion inmediata antes de seguir agregando funcionalidades al sistema.

Seccion	Critico	Alto	Medio	Bajo	Total
Pisos (PisoSectoresTab + api.ts)	4	5	9	7	25
Racks (OcupacionTab + TrasladoTab)	3	4	6	8	21
Infraestructura (sync, auth, kardex)	4	4	8	3	19
TOTAL	11	13	18	15	57

Tabla 1: Distribucion de hallazgos por seccion y severidad

1.1 Escala de Severidad

Nivel	Significado	Accion Requerida
CRITICO	Falla de seguridad, perdida de datos o corrupcion de inventario	Corregir inmediatamente. El sistema es vulnerable en produccion.
ALTO	Falla funcional que afecta operaciones criticas de negocio	Corregir en los proximos 1-3 dias. Impacta la operacion diaria.
MEDIO	Problema de calidad, rendimiento o experiencia de usuario	Planificar correccion dentro de la proxima semana.
BAJO	Cuestiones de estilo, limpieza de codigo o mejoras menores	Corregir cuando sea conveniente, sin urgencia.

Tabla 2: Escala de severidad utilizada en la auditoria

2. Hallazgos Criticos (Severidad CRITICO)

Los siguientes 11 hallazgos representan las vulnerabilidades y defectos mas graves identificados en el sistema. Cada uno de ellos tiene el potencial de causar perdida de datos, comprometer la seguridad de toda la aplicacion, o corromper el inventario de manera silenciosa. Se recomienda abordar estos problemas antes de continuar con cualquier desarrollo de nuevas funcionalidades.

2.1 Clave de Rol de Servicio Expuesta al Navegador

CRITICO

Archivos: supabase/client.ts:7, auth.ts:17, AuthGate.tsx:83

La clave de rol de servicio (service role key) de Supabase esta almacenada en una variable de entorno con el prefijo NEXT_PUBLIC_ (NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY). Esto significa que la clave se empaqueta dentro del JavaScript que se envia a cada navegador del usuario. Cualquier persona puede abrir las herramientas de desarrollador del navegador, localizar la clave, y usarla para eludir todas las politicas de seguridad a nivel de fila (RLS). La clave de servicio otorga acceso completo de administrador a todas las tablas, incluyendo la capacidad de eliminar datos, modificar perfiles de usuario, crear cuentas de administrador, y acceder a la API de administracion de GoTrue.

El impacto es devastador: un usuario malintencionado puede extraer la clave desde el bundle del cliente, usarla para otorgarse el rol de administrador, eliminar todos los movimientos, el catalogo o los datos de usuario, leer cualquier perfil de usuario, y crear cuentas de administrador ilimitadas. La variable dataClient (lineas 84-106 de client.ts) es un cliente Supabase del lado del navegador que utiliza la clave de rol de servicio, dando a cada visitante acceso completo de administrador a la base de datos.

Recomendacion: Mover la clave de rol de servicio a una variable de entorno solo del servidor (sin el prefijo NEXT_PUBLIC_) y enrutar todas las operaciones de dataClient a traves de rutas API de Next.js o funciones Edge de Supabase. El cliente anonimo junto con RLS deberia ser la unica ruta para las operaciones del lado del cliente.

2.2 Logica de Traslado con Excedente Crea Stock Fantasma

CRITICO

Archivos: PisoSectoresTab.tsx, lineas 982-1035 (ejecutarTrasladoPiso)

La logica de traslado en la seccion de Pisos maneja los items con excedente (donde la cantidad ingresada supera el stock actual) de manera incorrecta. El sistema primero realiza un traslado completo de la cantidad ingresada desde el origen al destino, y luego crea un ingreso adicional para la cantidad excedente en el destino. Por ejemplo, si un usuario ingresa cantidad=100 pero el stockActual=50, el origen recibe una salida de 100 (quedando en -50), el destino recibe un ingreso de 100 (por el traslado) mas un ingreso de 50 (por el excedente), totalizando 150. Esto genera un balance negativo en el origen y un stock duplicado en el destino, corrompiendo silenciosamente la integridad del inventario.

Recomendacion: Reescribir ejecutarTrasladoPiso para que nunca transfiera mas del stockActual. Limitar el traslado base al stock actual y agregar un ingreso unico para la cantidad excedente.

2.3 Operaciones No Atomicas en Registro de Movimientos

CRITICO

Archivos: api.ts, lineas 1190-1241 (registrarTrasladoPosicion), 1076-1117 (registrarIngresoPosicion), 1122-1150 (registrarSalidaPosicion), 1156-1184 (registrarDevolucionPosicion)

Todas las funciones de registro de movimientos en la API de Pisos ejecutan entre 2 y 4 llamadas separadas a la base de datos sin transaccion. La funcion registrarTrasladoPosicion, por ejemplo, crea el encabezado del movimiento de salida, luego el encabezado del movimiento de ingreso, luego inserta los detalles de salida, y finalmente los detalles de ingreso. Si alguna de estas llamadas falla a mitad de camino, la base de datos queda en un estado inconsistente con registros de encabezado huérfanos o registros de detalle parciales. No existe ningun mecanismo de reversión. Este patron se repite de manera identica en las funciones de ingreso, salida y devolucion.

Recomendacion: Envolver todas las operaciones de registro de movimientos en una funcion RPC de Supabase (por ejemplo, piso_registrar_movimiento) con una transaccion de base de datos, o utilizar funciones Edge de Supabase que soporten transacciones nativamente.

2.4 Filtrado de Columnas en listarMovimientos Esta Roto

CRITICO

Archivos: api.ts, lineas 446-461

La funcion listarMovimientos tiene una falla logica completa en el filtrado por columna. Cuando se proporciona columnaId pero no sectorId, la consulta usa columnaId como filtro de sector_id, lo cual nunca coincidira con nada. Adicionalmente, cuando la consulta si ejecuta, unicamente verifica si columnaId pertenece al sector pero nunca filtra los movimientos devueltos por columna. La funcion devuelve todos los datos o un array vacio, sin filtrado intermedio posible.

Recomendacion: Reescribir listarMovimientos para unir correctamente a traves de la cadena nivel-posicion-subcolumna-columna y aplicar filtros apropiados en cada nivel de la jerarquia.

2.5 Falta Signo \$ en Literal de Plantilla - CSS Roto

CRITICO

Archivos: TrasladoTab.tsx, linea 965

En la seccion de Racks, en el componente TrasladoTab, se encuentra un error de sintaxis en un literal de plantilla de JavaScript. El bloque condicional que aplica colores de fondo al indicador de ajuste carece del prefijo \$ antes de la llave de apertura, lo que significa que toda la expresion condicional (excedeStock ? ... : ...) se convierte en texto literal dentro del atributo className en lugar de ser evaluada. Como resultado, el indicador de ajuste se renderiza

con estilos rotos o invisibles, y el usuario nunca ve la codificación de color correcta (ambar para excedente, celeste para ajuste normal) durante las transferencias automáticas.

Recomendación: Cambiar `className={rounded-xl border p-3 }` por `className={rounded-xl border p-3 }$` agregando el signo `$` faltante.

2.6 Métrica Multi-Lote Permanentemente en Cero

CRÍTICO

Archivos: *OcupacionTab.tsx*, línea 112 (*calcularOcupacion*)

En la sección de Racks, la función *calcularOcupacion* asigna el valor `codigos.size` al campo *lotas*, pero *codigos* ya es `Array.from(codigos)`, lo que significa que *lotas* siempre será igual a `codigos.length`. El filtro de multi-lote en la línea 261 verifica `codigos.length === 1` y `lotas > 1` simultáneamente, lo cual es imposible porque si `codigos.length` es 1 entonces *lotas* es 1. Esto significa que la estadística de multi-lote en el dashboard, en el dashboard por bloque, en la exportación Excel, y en la leyenda, será permanentemente 0. Los usuarios no pueden identificar celdas con múltiples lotes del mismo producto con diferentes fechas de vencimiento, frustrando completamente el propósito del seguimiento FEFO (First Expired, First Out).

Recomendación: Modificar *calcularOcupacion* para rastrear el número de valores distintos de *f_vencimiento* por código, no solo el número de códigos distintos. Si la decisión de negocio es eliminar el seguimiento por lotes, eliminar los elementos de UI de multi-lote.

2.7 Campo Proveedor Perdido Durante Traslado

CRÍTICO

Archivos: *OcupacionTab.tsx*, líneas 533-541 (*ejecutarTraslado*)

La función *ejecutarTraslado* del componente *OcupacionTab* no incluye el campo proveedor en el objeto de entrada del traslado, a pesar de que el tipo *TrasladoInput* lo soporta y el ítem de origen (*trItem*) lo contiene. Esto significa que cada traslado realizado desde la pestaña de *Ocupacion* registrará el movimiento de destino sin información del proveedor. En contraste, la función *doTraslado* del componente *TrasladoTab* (línea 272) sí pasa correctamente proveedor: `origin.proveedor`. Este comportamiento inconsistente entre las dos rutas de traslado en la misma sección de Racks causa pérdida silenciosa de metadatos críticos en la base de datos.

Recomendación: Agregar `proveedor: trItem.proveedor` al objeto de entrada en *ejecutarTraslado*.

2.8 Definiciones de Tipos TypeScript Incompletas

CRÍTICO

Archivos: *supabase/types.ts*, líneas 433-486 (RPC types), 39-98 (movimientos table)

Las definiciones de tipos de TypeScript para los RPCs de Supabase estan gravemente incompletas. El tipo registrar_movimiento_kardex no declara los parametros p_uuid_sync y p_codigo_inc que el codigo efectivamente envia. De manera similar, registrar_traslado_kardex carece de p_uuid_sync y p_codigo_inc. El tipo de retorno de ambos RPCs dice { success: boolean; previous_stock: number; new_stock: number } pero el SQL real retorna JSONB con claves como origin_previous_stock y origin_new_stock. Ademas, la tabla movimientos carece de las columnas uuid_sync y codigo_inc en sus definiciones de tipo. Los roles de usuario definen 7 roles en auth.ts pero solo 2 en el enum app_role de supabase-types.ts. Estas discrepancias eliminan completamente la seguridad de tipos en tiempo de compilacion.

Recomendacion: Regenerar supabase-types.ts con el esquema actual de la base de datos, incluyendo uuid_sync, codigo_inc, p_uuid_sync, p_codigo_inc y los 7 roles de usuario.

2.9 Falta de Validacion de Cantidad en Alerta de Traslado

CRITICO

Archivos: OcupacionTab.tsx, lineas 528-530 (ejecutarTraslado)

La funcion ejecutarTraslado es invocada desde dos rutas diferentes: doTransferir (que valida qty > 0) y el boton de confirmacion del dialogo de alerta (que no realiza ninguna validacion). Si la cantidad llega como NaN o un valor negativo, se envia directamente al RPC que rechazara la solicitud con un error opaco del servidor en lugar de un mensaje claro del lado del cliente. Esto afecta directamente la funcionalidad de alerta de posicion ocupada que fue implementada recientemente.

Recomendacion: Agregar validacion de isNaN(qty) o qty <= 0 al inicio de ejecutarTraslado.

2.10 Traslado Fallback No Atomico Puede Perder Stock

CRITICO

Archivos: kardex.ts, lineas 575-660 (trasladarMovimientoFallback)

La funcion trasladarMovimientoFallback en la seccion de Racks inserta de 2 a 3 filas (ajuste, salida, traslado) en una sola llamada .insert(...). Sin embargo, PostgREST no ejecuta estas operaciones en una transaccion unica por defecto. Si la conexion se cae despues de escribir la salida pero antes de escribir el traslado (ingreso en destino), el stock desaparece: se resta del origen pero nunca se agrega en el destino. La ruta RPC (registrar_traslado_kardex) si usa advisory locks y una transaccion unica, pero la ruta fallback no tiene dicha proteccion.

Recomendacion: Migrar toda la logica de traslado al RPC con transaccion y eliminar el fallback directo, o agregar advisory locks al fallback.

2.11 Duplicacion de uuid_sync en RPC con Ajuste

CRITICO

Archivos: supabase/migrations/20260620_rpc_inc_stock_fix.sql, lineas 183-213

El RPC registrar_traslado_kardex asigna p_uuid_sync a las tres inserciones (ajuste, salida, traslado). La columna uuid_sync tiene un indice unico parcial (WHERE uuid_sync IS NOT NULL). Si se proporciona p_uuid_sync y p_cantidad_ajuste es diferente de 0, el RPC fallara con una violacion de constraint unico porque intenta insertar 3 filas con el mismo uuid_sync no nulo. Esto significa que los traslados sincronizados offline con ajustes siempre fallaran en la base de datos. El fallback de TypeScript (kardex.ts) correctamente asigna uuid_sync solo a una fila, pero el RPC no.

Recomendacion: Modificar el RPC para asignar p_uuid_sync unicamente a la primera fila (el ajuste) y NULL a las demas, igual que el fallback de TypeScript.

3. Hallazgos de Alta Severidad

Los siguientes 13 hallazgos representan fallas funcionales que afectan directamente las operaciones criticas de negocio. Si bien no comprometen la seguridad del sistema en su totalidad, pueden causar operaciones duplicadas, datos inconsistentes, y errores silenciosos que impactan la operacion diaria del almacen.

ID	Severidad	Ubicacion	Problema	Descripcion	Accion Recomendada
H1	ALTO	PisoSectoresTab.tsx:605	Reset de nivel en traslado/devolucion	openTraslado y openDevolucion reinician selectedNivelId al primer nivel, rompiendo la seleccion del usuario.	Eliminar set de selectedNivelId en openTraslado y openDevolucion.
H2	ALTO	PisoSectoresTab.tsx:788	IDs manuales sin resolver pasan a la API	ensureManualBloqueCreated empuja filas con prefijo manual_ si la creacion y busqueda fallan.	Lanzar error si un bloque manual no puede resolverse en vez de pasar ID roto.
H3	ALTO	Pisos + Racks	Doble clic causa operaciones duplicadas	Ninguna funcion doXxx verifica busy al inicio. El estado de React es asincrono, permite doble envio.	Agregar if (busy) return usando un ref-based guard pattern.
H4	ALTO	api.ts:1285	stockPorNivelPosicion siempre retorna bloque_codigo vacio	La funcion agrega stock por nivel_id pero descarta informacion por bloque. bloque_codigo es siempre vacio.	Reescribir para rastrear stock por (nivel_id, bloque_id), no solo por nivel_id.
H5	ALTO	api.ts (multiple)	Duplicacion masiva de codigo FEFO	stockDetalleNivel (120 lineas) es casi identico al fallback de stockDetallePosicion.	Extraer logica FEFO compartida en una funcion helper.

H6	ALTO	OcupacionTab.ts:375	Falta validacion de descripcion y un en ingreso	doIngreso y doIngresoINC validan codigo y cantidad pero no descripcion ni unidad.	Agregar validacion de campos requeridos antes de enviar al RPC.
H7	ALTO	auth.ts:205-271	Sin checks de autorizacion en funciones de perfil	getTodosLosPerfiles, cambiarAprobado, cambiarRol, eliminarPerfil no verifican roles del usuario.	Agregar verificacion de rol admin en cada funcion o restringir a rutas API del servidor.
H8	ALTO	sync-engine.ts:248	Errores silenciados en SyncEngine	Multiples catch blocks ocultan errores sin registro: refreshCounts, syncAll, etc.	Implementar logging estructurado y notificacion al usuario de errores de sincronizacion.
H9	ALTO	catalogo.ts:228	clearPisoBloques no es atomico	Si falla despues de eliminar asignaciones de columna pero antes de eliminar bloques, queda estado parcial.	Envolver en RPC de base de datos con transaccion.
H10	ALTO	sync-engine.ts:202	Ping filtra estructura de base de datos	La funcion ping confirma a atacantes que la tabla perfiles existe y la API REST es accesible.	Remover ping o limitar a endpoints no reveladores de estructura.
H11	ALTO	offline-db.ts:13	DB_VERSION nunca incrementado	Si el esquema IndexedDB necesita evolucionar, los datos existentes no se migran.	Implementar sistema de versiones con onupgradeneeded para migraciones.
H12	ALTO	TrasladoTab.ts:250	Variable shadowing de movs	La desestructuracion { movs } del response sombrea el estado movs del componente.	Renombrar a { movs: updatedMovs } para evitar confusion.
H13	ALTO	OcupacionTab.ts:571	Doble setActionBusy causa flicker de boton	El flujo de alerta resetea busy y luego el finally lo resetea de nuevo, causando flicker visual.	Refactorizar control de flujo para un solo punto de activacion/deactivacion.

Tabla 3: Hallazgos de alta severidad

4. Hallazgos de Severidad Media

Los hallazgos de severidad media afectan la calidad del codigo, el rendimiento del sistema, y la experiencia del usuario sin causar fallas criticas de negocio. Sin embargo, su correccion mejora significativamente la robustez y

mantenibilidad del sistema a largo plazo.

ID	Sev.	Ubicacion	Problema	Describeion
M1	ME DI O	api.ts:940	Items sin fecha de vencimiento ordenados primero en FEFO	Los items sin fecha de vencimiento se consumen primero, pero deberian consumirse ultimo si no tienen caducidad.
M2	ME DI O	PisoSectoresTab.tsx:1156	doMassSalida ejecuta posiciones en paralelo sin rollback	Promise.all procesa todas las posiciones concurrentemente sin mecanismo de deshacer si algunas fallan.
M3	ME DI O	PisoSectoresTab.tsx:912	Seleccion de nivel en salida puede producir nivel_id incorrecto	Cuando salNivelTab es all, usa selectedNivelId que apunta al nivel visualizado, no necesariamente donde esta el stock.
M4	ME DI O	api.ts (multiple)	Console.log de debug en produccion	Multiples sentencias console.log con prefijo [Piso] quedaron en el codigo de produccion.
M5	ME DI O	api.ts:206	eliminarBloque no verifica error en eliminacion de relaciones	Si la eliminacion de piso_columna_bloques falla, se eliminan los bloques pero quedan relaciones huérfanas.
M6	ME DI O	api.ts (traslado)	Traslado crea dos movimientos sin vincular	Salida e ingreso son registros independientes sin campo traslado_id para auditar correspondencia.
M7	ME DI O	PisoSectoresTab.tsx:422	useEffect causa refetches innecesarios	Cualquier cambio en dependencias de useCallbacks dispara la recarga de sectores, posiciones y bloques.
M8	ME DI O	TrasladoTab.tsx:93	Actualizaciones realtime invalidan selectedOrigin	useMovimientosRealtime actualiza movs pero no locations, causando datos obsoletos.
M9	ME DI O	OcupacionTab.tsx:464	Variable detail sombrea estado detail en doSalida	Un const detail local en el catch sombrea el estado detail del componente, creando confusion.
M10	ME DI O	sync-engine.ts:639	offlineAwareTraslado sin passthrough de uuidSync	Siempre genera nuevo UUID, diferente de offlineAwareAddMovimiento que acepta uuidSync opcional.

MI 1	ME DI O	kardex.ts:607	uuid_sync solo en una fila del traslado fallback	Solo el ajuste tiene uuid_sync, la salida y el traslado tienen NULL. Diferente del RPC que lo pone en todas.
MI 2	ME DI O	kardex.ts:50	fromRow usa casts inseguros sin validacion	Tipo y turno se casteo con as sin verificar que el valor sea valido en tiempo de ejecucion.
MI 3	ME DI O	kardex.ts:73	fetchMovimientos descarga tabla completa despues de cada escritura	Cada operacion de escritura dispara una descarga completa de la tabla movimientos con paginacion.
MI 4	ME DI O	useConnectivity.ts:26	SyncEngine nunca se destruye en unmount	Los event listeners y intervalos de ping y conteos persisten despues de desmontar el componente.
MI 5	ME DI O	sync-engine.ts:606	Cola offline no valida datos al encolar	Se aceptan movimientos con codigo vacio, cantidad negativa, o ubicaciones invalidas.
MI 6	ME DI O	sync-engine.ts:545	getConflicts escanea todos los movimientos pendientes	$O(n)$ donde podria ser $O(k)$ con un indice IndexedDB por status.
MI 7	ME DI O	catalogo.ts:13	Cache a nivel de modulo puede filtrar en SSR	Variables _cache y _cacheLoaded pueden persistir datos obsoletos en contextos de servidor.
MI 8	ME DI O	prisma/schema.prisma	Schema Prisma es boilerplate residual	Modelos User y Post de Next.js generico, no usados. db.ts conecta a SQLite no utilizado.

Tabla 4: Hallazgos de severidad media

5. Hallazgos de Baja Severidad

Los hallazgos de baja severidad son cuestiones de limpieza de codigo, estilo, y mejoras menores que no afectan la funcionalidad del sistema pero que deben abordarse para mejorar la calidad general del codigo y facilitar el mantenimiento futuro.

ID	Sev.	Ubicacion	Problema
L1	BAJO	PisoSectoresTab.tsx:39	Import Checkbox sin usar
L2	BAJO	PisoSectoresTab.tsx:1247	Componentes NivelSelector definidos dentro del componente principal

<i>L3</i>	<i>BAJO</i>	<i>api.ts (multiple)</i>	<i>Type assertions inseguros (as unknown[])</i>
<i>L4</i>	<i>BAJO</i>	<i>api.ts:466</i>	<i>calcularStockNivel no distingue lotes por fecha</i>
<i>L5</i>	<i>BAJO</i>	<i>PisoSectoresTab.tsx:10</i>	<i>Import obtenerPrimerNivel potencialmente sin usar</i>
<i>L6</i>	<i>BAJO</i>	<i>api.ts:216</i>	<i>reemplazarCatalogoBloques usa neq(id,) frágil</i>
<i>L7</i>	<i>BAJO</i>	<i>OcupacionTab.tsx:5</i>	<i>Import fetchOcupacionCeldas solo usado en fallback</i>
<i>L8</i>	<i>BAJO</i>	<i>OcupacionTab.tsx:240</i>	<i>refreshDetail traga errores silenciosamente</i>
<i>L9</i>	<i>BAJO</i>	<i>sync-engine.ts:248</i>	<i>refreshCounts falla silenciosamente</i>
<i>L10</i>	<i>BAJO</i>	<i>OcupacionTab.tsx:383</i>	<i>Sin validacion de fecha cuando ingSinFecha es false</i>
<i>L11</i>	<i>BAJO</i>	<i>OcupacionTab.tsx:602</i>	<i>Export usa lookup O(n2)</i>
<i>L12</i>	<i>BAJO</i>	<i>OcupacionTab.tsx:614</i>	<i>Non-null assertion cell! en export</i>
<i>L13</i>	<i>BAJO</i>	<i>auth.ts:192,218</i>	<i>Logica de extraccion de roles duplicada e inconsistente</i>
<i>L14</i>	<i>BAJO</i>	<i>kardex.ts:465</i>	<i>stockEnUbicacion retorna [] en todo error</i>
<i>L15</i>	<i>BAJO</i>	<i>ConnectionIndicator.tsx:141</i>	<i>max-w duplicado y conflictivo</i>

Tabla 5: Hallazgos de baja severidad

6. Plan de Remediation por Prioridad

A continuacion se presenta el plan de remediation organizado por prioridad temporal. Se recomienda abordar los problemas en el orden indicado, comenzando por los que representan mayor riesgo para la seguridad e integridad del sistema. Cada fase incluye una estimacion del esfuerzo requerido y los recursos necesarios para su implementacion.

6.1 Fase Inmediata (Dias 1-2)

Esta fase debe completarse antes de cualquier otro desarrollo. Los items aqui incluidos representan riesgos activos de seguridad y corrupcion de datos que estan expuestos en el ambiente de produccion actual. El esfuerzo estimado es de 2 a 3 dias de desarrollo concentrado, preferiblemente por un desarrollador senior con conocimiento completo de la arquitectura del sistema.

#	Hallazgo	Accion	Esfuerzo
1	Clave service role expuesta	Mover a variable de entorno solo servidor y crear rutas API	1 dia
2	Logica de excedente en traslado	Reescribir ejecutarTrasladoPiso con cap de stock actual	2 horas

3	Operaciones no atomicas	Crear RPC piso_registrar_movimiento con transaccion	1 dia
4	Filtrado de columnas roto	Reescribir listarMovimientos con joins correctos	3 horas
5	CSS roto en TrasladoTab	Agregar signo \$ faltante en template literal	5 minutos
6	Multi-lote en cero	Modificar calcularOcupacion para tracking por fecha de vencimiento	2 horas
7	Proveedor perdido en traslado	Agregar proveedor: trItem.proveedor en ejecutarTraslado	5 minutos
8	Validacion de cantidad en alerta	Agregar validacion isNaN/qty<=0 al inicio de ejecutarTraslado	10 minutos

Tabla 6: Acciones de fase inmediata

6.2 Fase Corto Plazo (Semana 1)

Esta fase aborda los problemas funcionales que afectan las operaciones diarias del almacen. Los items aqui listados no representan riesgos de seguridad inmediatos, pero si pueden causar frustracion en los usuarios, operaciones duplicadas, y datos inconsistentes que requieren correccion manual. El esfuerzo estimado es de 3 a 4 dias de desarrollo.

#	Hallazgo	Accion	Esfuerzo
1	Doble clic duplicado	Agregar ref-based guard en todas las funciones doXxx	2 horas
2	Reset de nivel en traslado	Eliminar set de selectedNivelId en openTraslado/openDevolucion	15 minutos
3	IDs manuales sin resolver	Lanzar error en vez de pasar ID roto en ensureManualBloqueCreated	10 minutos
4	stockPorNivelPosicion vacio	Reescribir para rastrear stock por bloque, no solo por nivel	2 horas
5	Validacion faltante en ingreso	Agregar validacion de descripcion y un en doIngreso/doIngresoINC	30 minutos
6	Sin autorizacion en perfiles	Agregar verificacion de rol admin en funciones de auth.ts	2 horas
7	Errores silenciados en SyncEngine	Implementar logging estructurado en catch blocks	1 hora
8	DB_VERSION sin incrementar	Implementar sistema de migracion de IndexedDB	3 horas

Tabla 7: Acciones de corto plazo

6.3 Fase Mediano Plazo (Semanas 2-3)

La fase de mediano plazo aborda la calidad general del codigo, la mantenibilidad, y el rendimiento del sistema. Estos cambios no son urgentes pero representan inversiones significativas en la salud tecnica del proyecto. Se recomienda planificarlos como parte del backlog regular de mantenimiento tecnico y abordarlos incrementalmente durante los sprints de desarrollo.

#	Hallazgo	Accion	Esfuerzo
1	Regenerar tipos TypeScript	Ejecutar supabase gen types con esquema actualizado	1 hora
2	Duplicacion de codigo FEFO	Extraer logica compartida en funcion helper	2 horas
3	Traslado fallback no atomico	Migrar toda logica al RPC con transaccion	1 dia

4	uuid_sync duplicado en RPC	Modificar RPC para asignar solo a primera fila	30 minutos
5	Console.log en produccion	Eliminar todas las sentencias de debug	30 minutos
6	Refactorizar clearPisoBloques	Crear RPC con transaccion para limpieza de bloques	1 hora
7	SyncEngine lifecycle	Implementar destroy() en useConnectivity	1 hora
8	Fetch de tabla completa	Reemplazar con queries dirigidas o suscripciones realtime	2 dias

Tabla 8: Acciones de mediano plazo

7. Conclusiones y Observaciones Generales

La auditoria revela que el sistema Rackly, a pesar de ser funcional y servir adecuadamente a sus usuarios actuales, presenta un numero significativo de vulnerabilidades de seguridad y defectos de integridad de datos que requieren atencion prioritaria. El hallazgo mas critico, la exposicion de la clave de rol de servicio al navegador del cliente, representa un riesgo de seguridad que debe corregirse inmediatamente independientemente de cualquier otra consideracion. Este problema por si solo justifica una interrupcion del desarrollo de nuevas funcionalidades hasta ser resuelto.

En cuanto a la integridad de los datos de inventario, los problemas encontrados en la logica de traslados (stock fantasma, balance negativo, operaciones no atomicas) son particularmente preocupantes en un sistema de gestion de inventario donde la precision de los datos es fundamental. La ausencia de transacciones en las operaciones de registro de movimientos significa que cualquier fallo de red o de base de datos a mitad de una operacion puede dejar el sistema en un estado que requiere intervencion manual para ser corregido, lo cual es inaceptable para un sistema de produccion.

Las condiciones de carrera por doble clic y la falta de guards basados en refs en las funciones de movimiento son problemas relativamente simples de corregir pero que pueden causar operaciones duplicadas que son dificiles de detectar y revertir una vez ocurridas. Se recomienda implementar estas correcciones como parte de la fase inmediata junto con los problemas criticos de seguridad.

A pesar de los problemas identificados, la arquitectura general del sistema muestra decisiones acertadas como la implementacion de sincronizacion offline con IndexedDB, el mecanismo de idempotencia basado en uuidSync, la clasificacion de errores en el sync engine, y el uso de RPCs con advisory locks para las operaciones principales de Racks. Estos fundamentos solidos hacen que las correcciones propuestas sean factibles sin requerir una re-arquitectura completa del sistema.